

Teaching Exam

Subject – Computer Science

**Data Structure and
Algorithm**



Lecture No –5

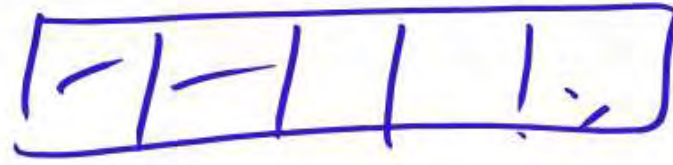
By–Satyendra Chaudhary Sir

Topics to be Covered

Topic

linked Lists, Trees.



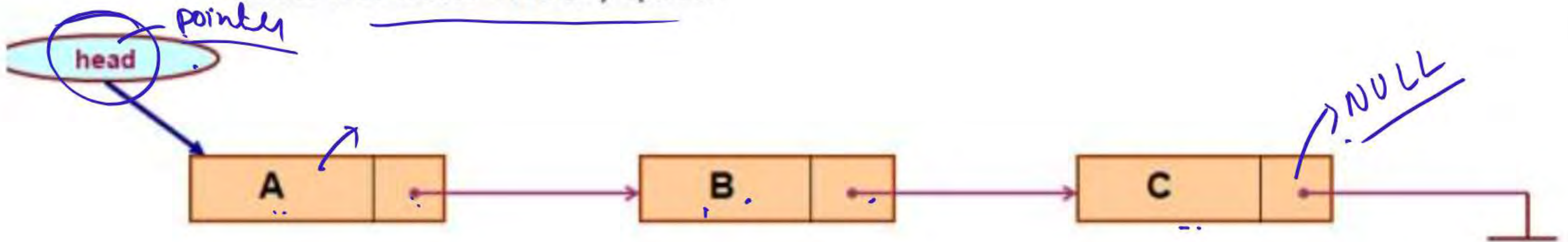


5 spaces

9

Introduction

- A linked list is a data structure which can change during execution.
 - Successive elements are connected by pointers.
 - Last element points to **NULL**.
 - It can grow or shrink in size during execution of a program.
 - It can be made just as long as required.
 - It does not waste memory space.



- Keeping track of a linked list:
 - Must know the pointer to the first element of the list (called start, head, etc.).
- Linked lists provide flexibility in allowing the items to be rearranged efficiently.
 - Insert an element. ✓
 - Delete an element. ✓

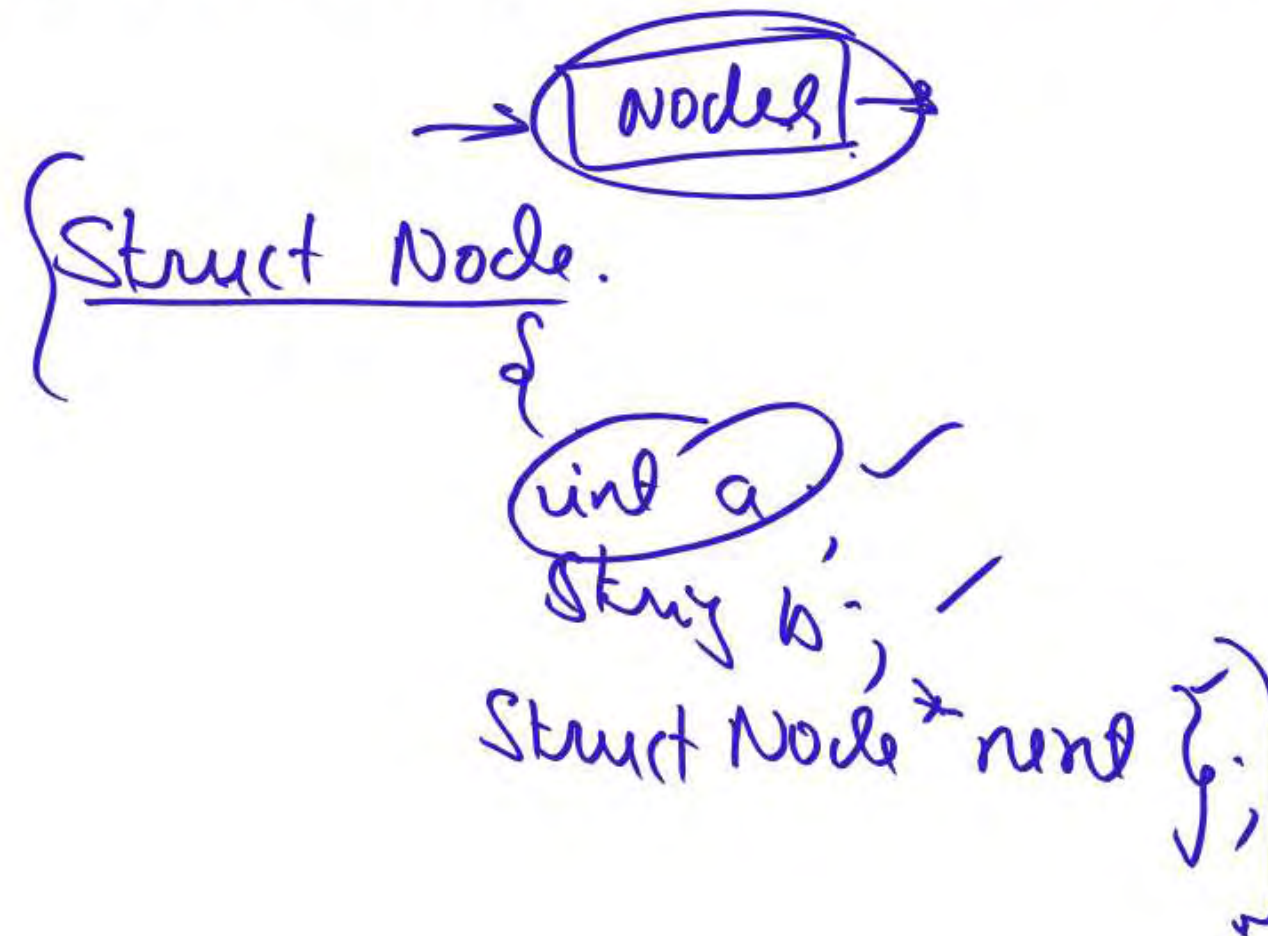
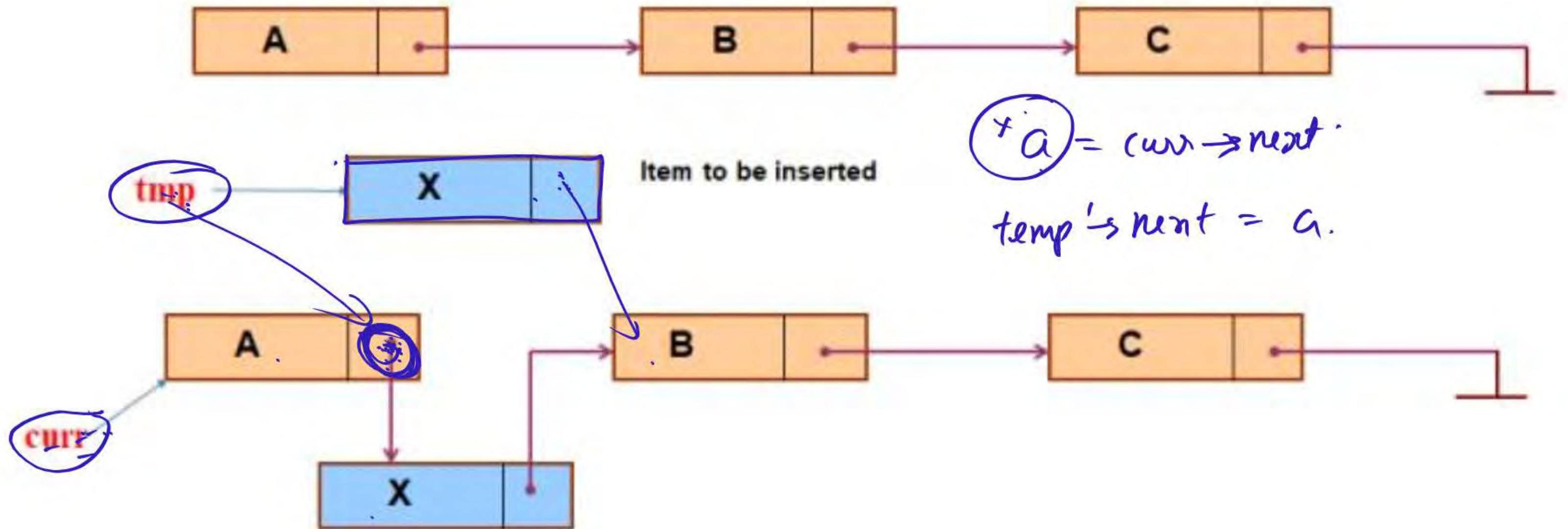


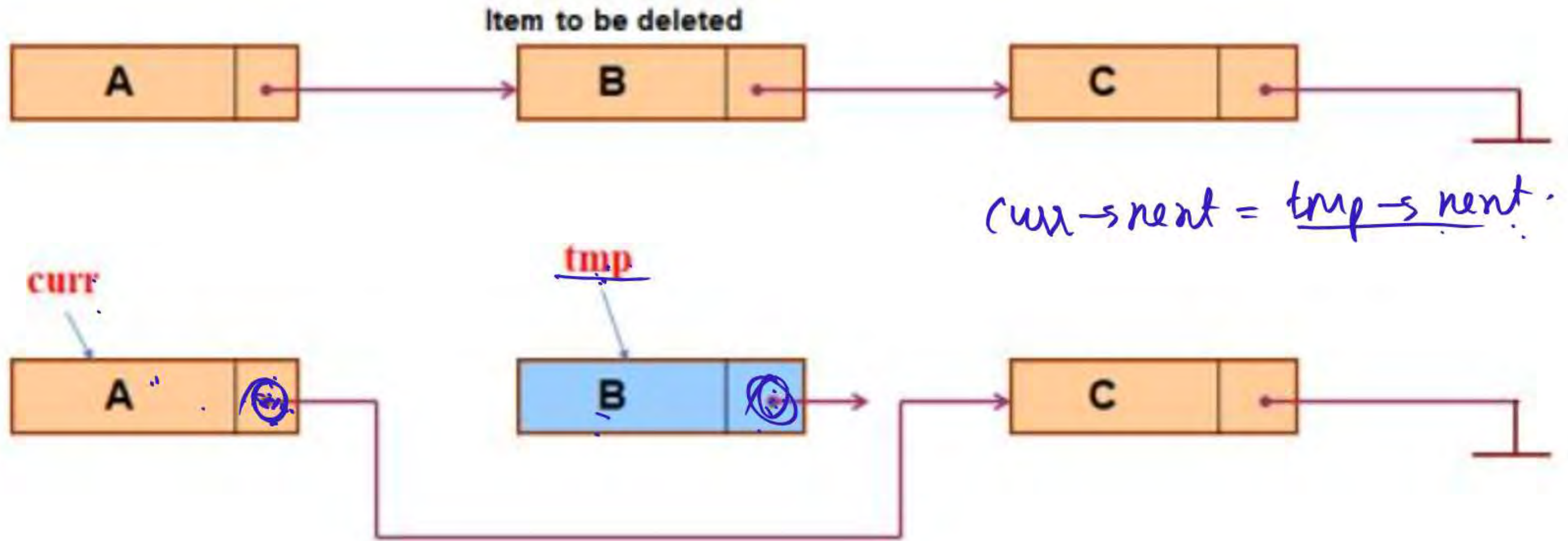
Illustration: Insertion



Pseudo-code for insertion

```
typedef struct nd {  
    struct item data;  
    struct nd * next;  
} node;  
  
void insert(node *curr)  
{  
    node * tmp;  
  
    tmp=(node *) malloc(sizeof(node));  
    tmp->next=curr->next;  
    curr->next=tmp;  
}
```

Illustration: Deletion

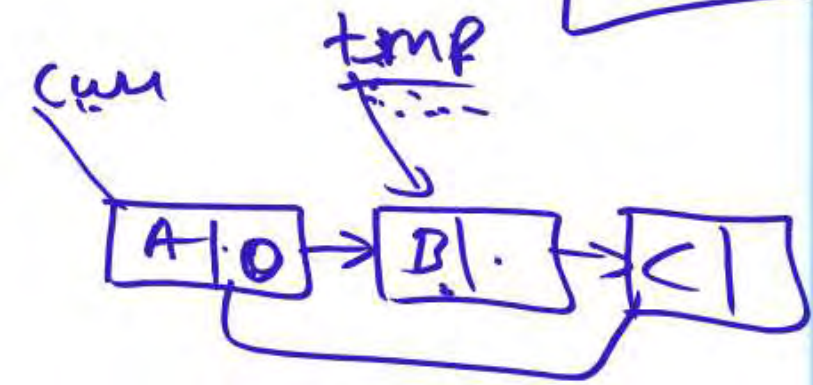


Pseudo-code for deletion

```
typedef struct nd {
    struct item data;
    struct nd * next;
} node;

void delete(node *curr)
{
    node * tmp;
    tmp = curr->next;
    curr->next = tmp->next;
    free(tmp);
}
```

0(1)



(curr->next = tmp->next)

In essence ...

- For insertion:
 - A record is created holding the new item.
 - The **next** pointer of the new record is set to link it to the item which is to follow it in the list.
 - The **next** pointer of the item which is to precede it must be modified to point to the new item.
- For deletion:
 - The **next** pointer of the item immediately preceding the one to be deleted is altered, and made to point to the item following the deleted item.

Array versus Linked Lists

- Arrays are suitable for:

$O(n)$

- Inserting/deleting an element at the end.
- Randomly accessing any element.
- Searching the list for a particular value.

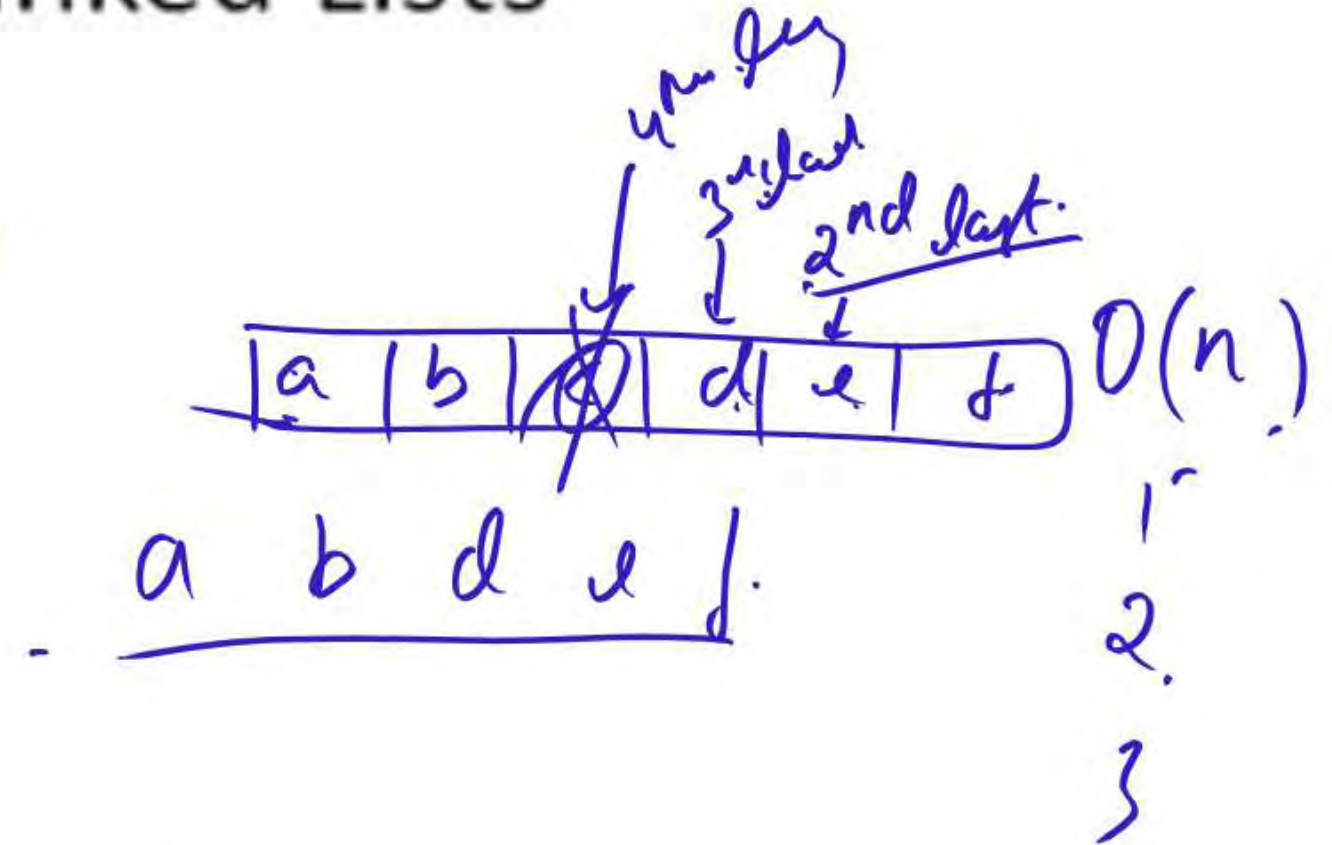
- Linked lists are suitable for:

$O(1)$

- Inserting an element.
- Deleting an element.

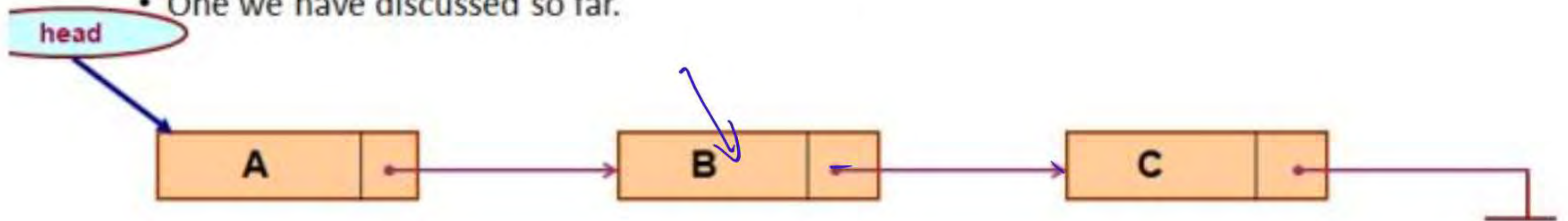
$O(1)$

- Applications where sequential access is required.
- In situations where the number of elements cannot be predicted beforehand.



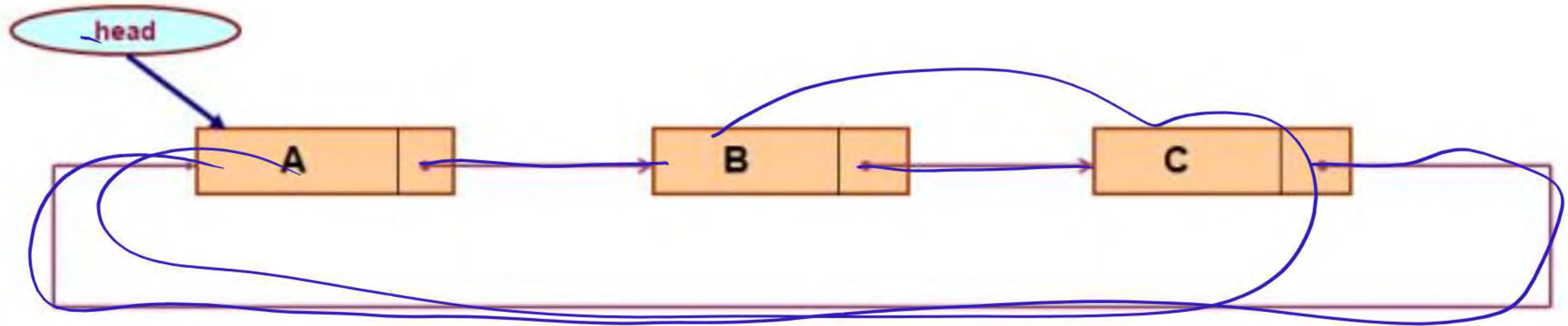
Types of Lists

- Depending on the way in which the links are used to maintain adjacency, several different types of linked lists are possible.
 - Linear singly-linked list (or simply linear list)
 - One we have discussed so far.

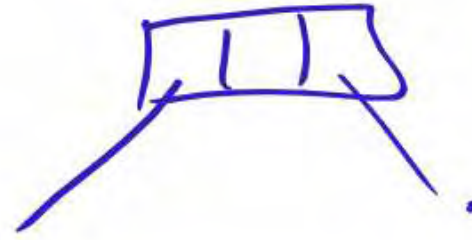


– Circular linked list

- The pointer from the last element in the list points back to the first element.

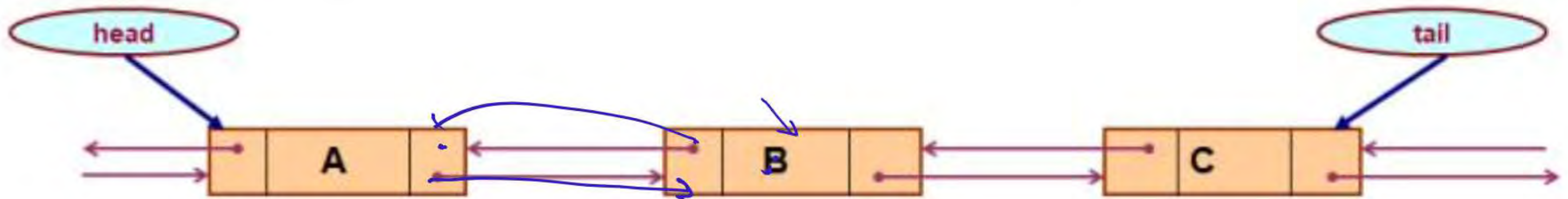


Struct Node* left; ;
Struct Node* right;



– Doubly linked list

- Pointers exist between adjacent nodes in both directions.
- The list can be traversed either forward or backward.
- Usually two pointers are maintained to keep track of the list, *head* and *tail*.



Basic Operations on a List

- Creating a list
- Traversing the list
- Inserting an item in the list
- Deleting an item from the list
- Concatenating two lists into one

List is an Abstract Data Type

- What is an abstract data type?
 - It is a data type defined by the user.
 - Typically more complex than simple data types like *int*, *float*, etc.
- Why abstract?
 - Because details of the implementation are **hidden**.
 - When you do some operation on the list, say insert an element, you just call a function.
 - Details of how the list is implemented or how the insert function is written is no longer required.

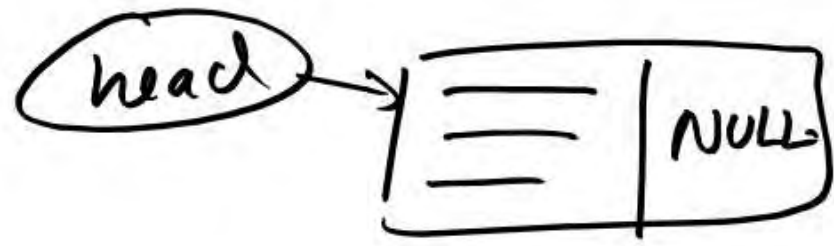
Example: Working with linked list

- Consider the structure of a node as follows:

struct stud { int roll;
int roll;
char name[25];
int age;
struct stud *next;
};

/* A user-defined data type called "node" */

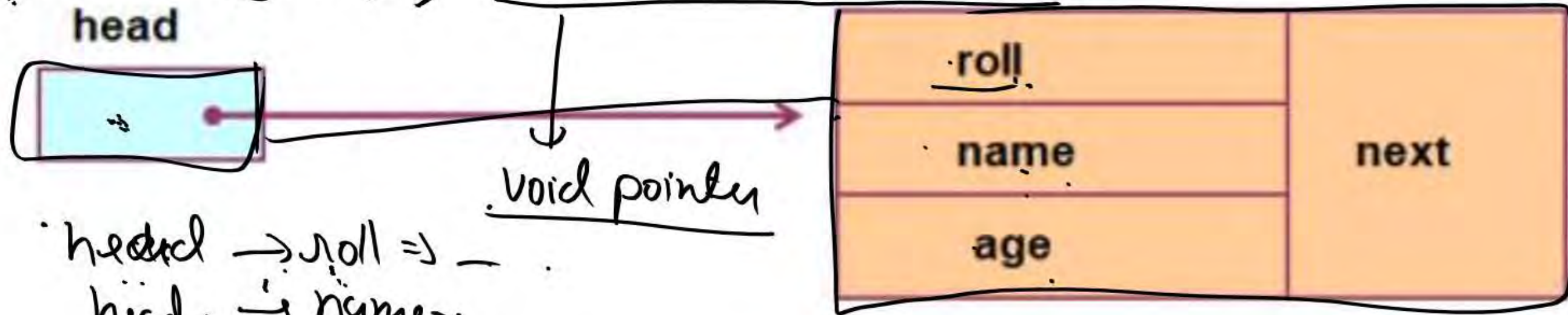
typedef struct stud node;
node *head;



How to begin?

- To start with, we have to create a node (the first node), and make **head** point to it.

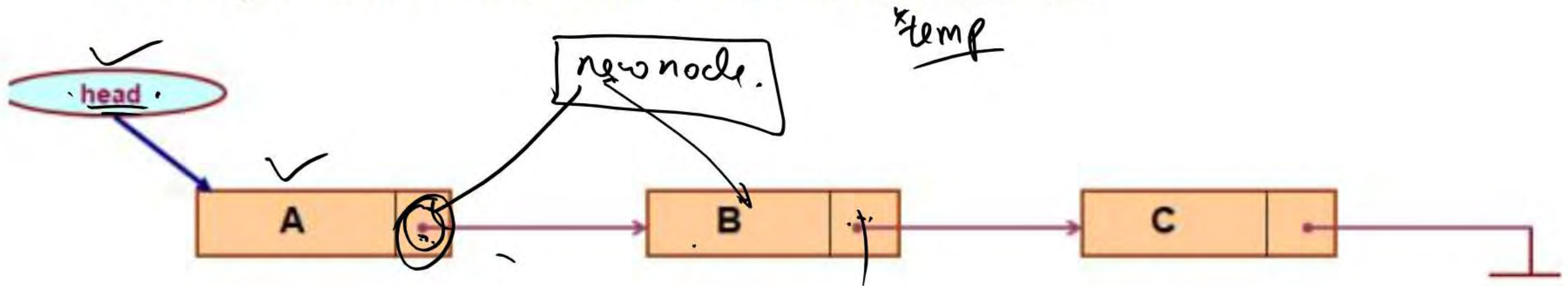
new node.
~~head~~ = (node *) malloc(sizeof(node));
 int * a; (stud *) malloc(sizeof(node));



head → roll ⇒
 head → name ⇒
 head → *next = NULL

Contd.

- If there are n number of nodes in the initial linked list:
 - Allocate n records, one by one.
 - Read in the fields of the records.
 - Modify the links of the records so that the chain is formed.




```
node *create_list()
{
```

node. {

```
int k, n;
node *p, *head;
```

```
printf ("\n How many elements to enter?");
scanf ("%d", &n);
```

```
for (k=0; k<n; k++)
{
```

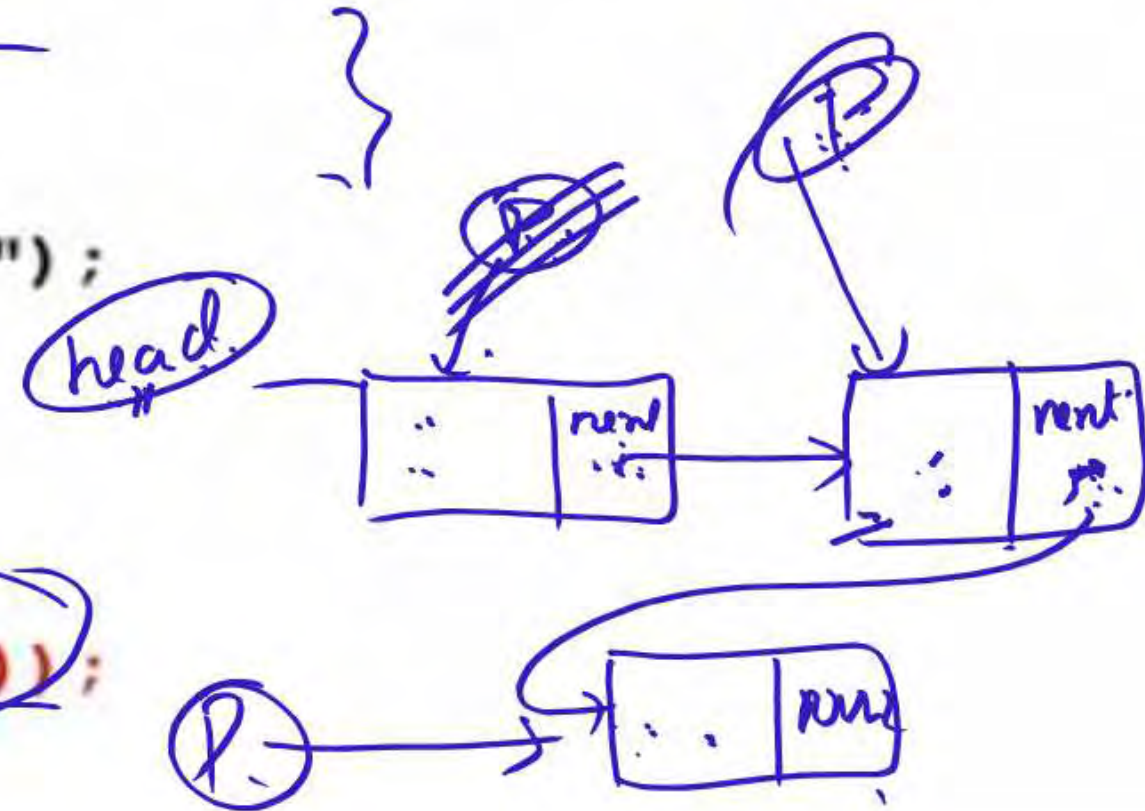
```
if (k == 0) {
    head = (node *) malloc(sizeof(node));
    p = head;
```

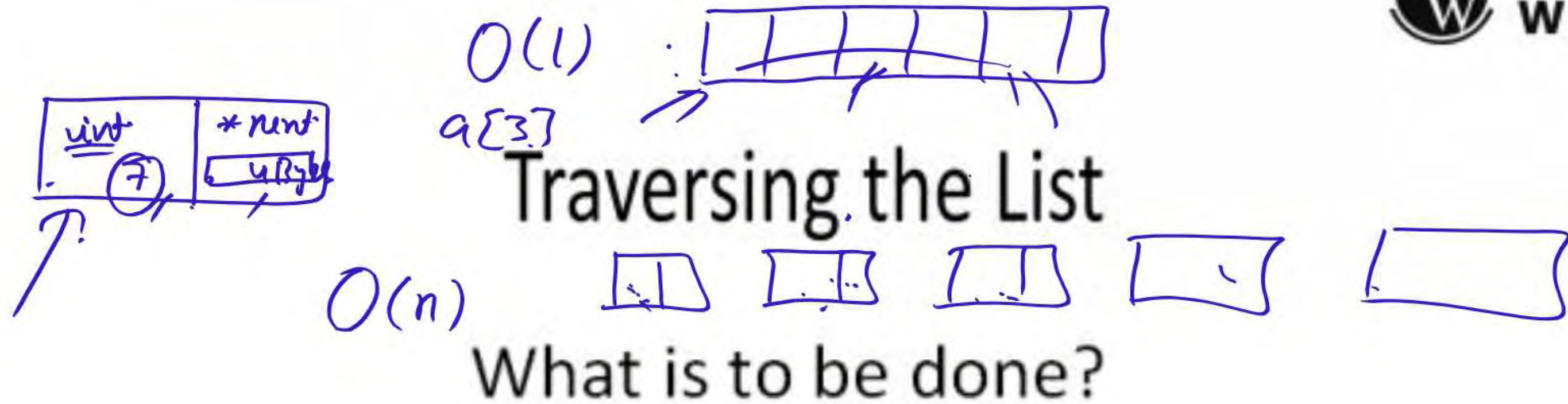
```
}
else {
```

```
    p->next = (node *) malloc(sizeof(node));
    p = p->next;
}
```

```
    scanf ("%d %s %d", &p->roll, p->name, &p->age);
}
```

```
p->next = NULL;
return head;
```





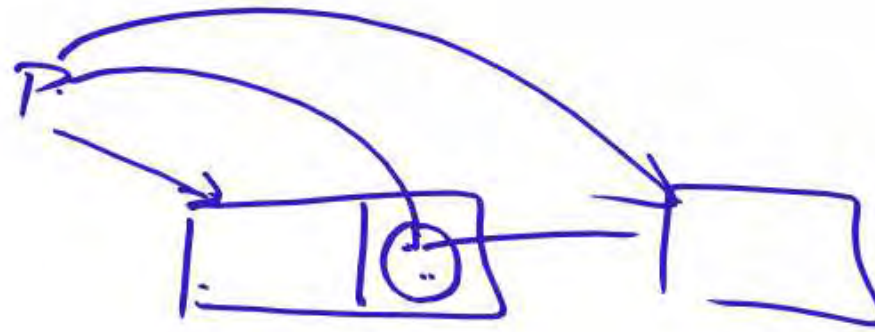
- Once the linked list has been constructed and *head* points to the first node of the list,
 - Follow the pointers.
 - Display the contents of the nodes as they are traversed.
 - Stop when the *next* pointer points to **NULL**.


```
void display (node *head)
```

```
{
    int count = 1;
    node *p;

    p = head;
    while (p != NULL)
    {
        printf ("\nNode %d: %d %s %d", count,
                p->roll, p->name, p->age);

        count++;
        p = p->next;
    }
    printf ("\n");
}
```



Inserting a Node in a List

How to do?

- The problem is to insert a node *before a specified node*.
 - Specified means some value is given for the node (called *key*).
 - In this example, we consider it to be *roll*.
- Convention followed:
 - If the value of roll is given as *negative*, the node will be inserted at the *end* of the list.

- When a node is added at the beginning,
 - Only one next pointer needs to be modified.
 - *head* is made to point to the new node.
 - New node points to the previously first element.
- When a node is added at the end,
 - Two next pointers need to be modified.
 - Last node now points to the new node.
 - New node points to **NULL**.
- When a node is added in the middle,
 - Two next pointers need to be modified.
 - Previous node now points to the new node.
 - New node points to the next node.

```
void insert (node **head)
{
    int k = 0, rno;
    node *p, *q, *new;

    new = (node *) malloc(sizeof(node));

    printf ("\nData to be inserted: ");
    scanf ("%d %s %d", &new->roll, new->name, &new->age);
    printf ("\nInsert before roll (-ve for end):");
    scanf ("%d", &rno);

    p = *head;

    if (p->roll == rno)                /* At the beginning */
    {
        new->next = p;
        *head = new;
    }
```



```

else
{
    while ((p != NULL) && (p->roll != rno))
    {
        q = p;
        p = p->next;
    }

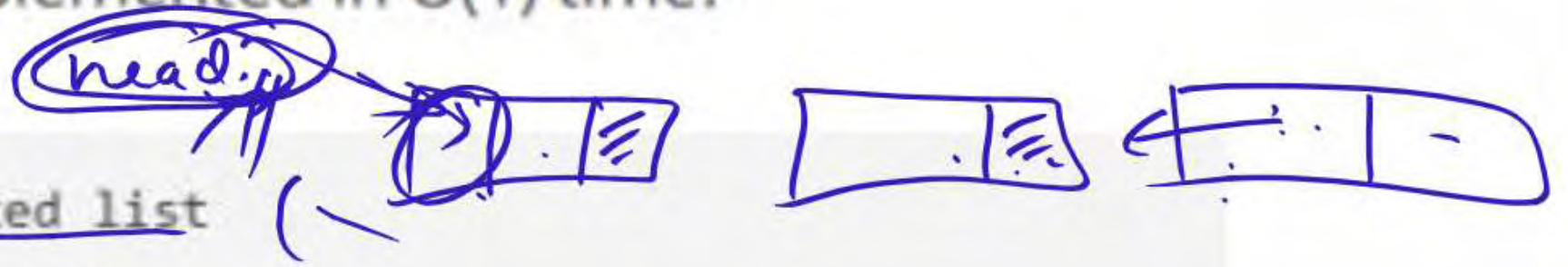
    if (p == NULL)                /* At the end */
    {
        q->next = new;
        new->next = NULL;
    }
    else if (p->roll == rno)
        /* In the middle */
        {
            q->next = new;
            new->next = p;
        }
    }
}

```

1. A linear collection of data elements where the linear node is given by means of pointer is called?

- a) Linked list //
- b) Node list .
- c) Primitive list .
- d) Unordered list .

2. Consider an implementation of unsorted singly linked list. Suppose it has its representation with a head pointer only. Given the representation, which of the following operation can be implemented in $O(1)$ time?



- i) Insertion at the front of the linked list
- ii) Insertion at the end of the linked list
- iii) Deletion of the front node of the linked list
- iv) Deletion of the last node of the linked list



- a) I and II
- b) I and III
- c) I, II and III
- d) I, II and IV

3. In linked list each node contains a minimum of two fields. One field is data field to store the data second field is?

- a) Pointer to character
- b) Pointer to integer
- c) Pointer to node
- d) Node

4. What would be the asymptotic time complexity to add a node at the end of singly linked list, if the pointer is initially pointing to the head of the list?

a) $O(1)$

b) $O(n)$

c) $\theta(n)$

d) Both $O(n)$ and $\theta(n)$

$2n$
 $3n$

head

$\frac{n}{2}$

n

$O(1)$

$O(2)$

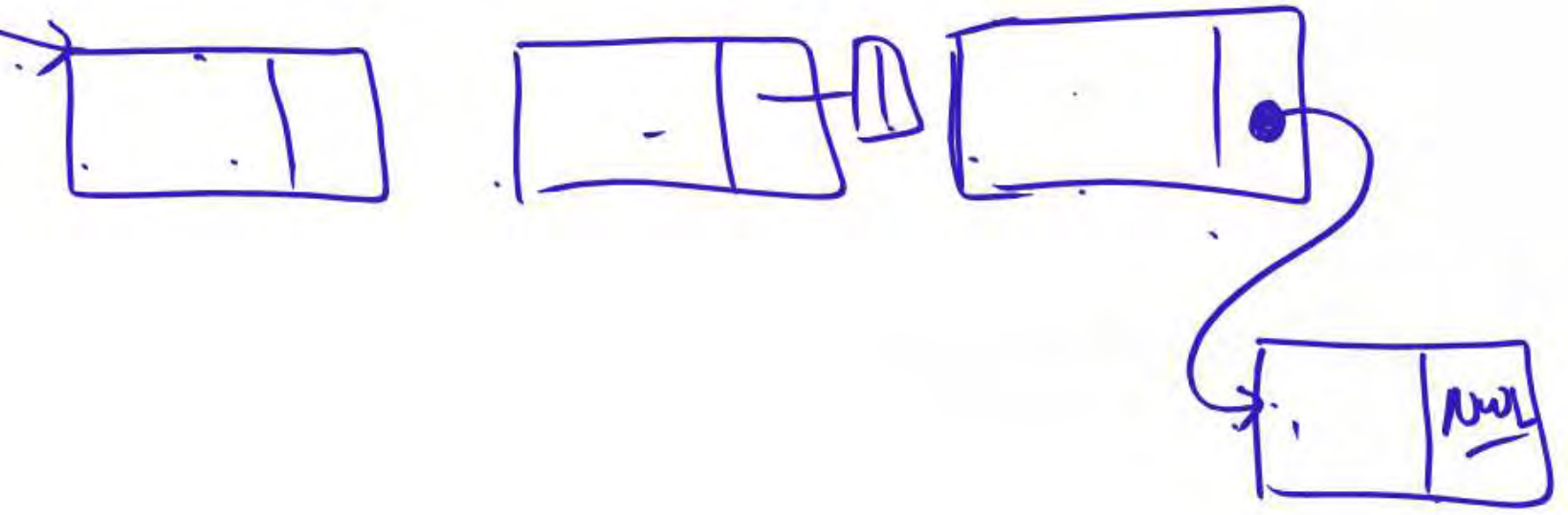
$O(3)$

$O(2n)$

$O(3n)$

$O(\frac{n}{2})$

$O(n)$



5. What would be the asymptotic time complexity to insert an element at the front of the linked list (head is known)?

- a) $O(1)$ ✓
- b) $O(n)$
- c) $O(n^2)$
- d) $O(n^3)$

6. What would be the asymptotic time complexity to find an element in the linked list?

a) $O(1)$

b) $O(n)$ ✓

c) $O(n^2)$

d) $O(n^4)$

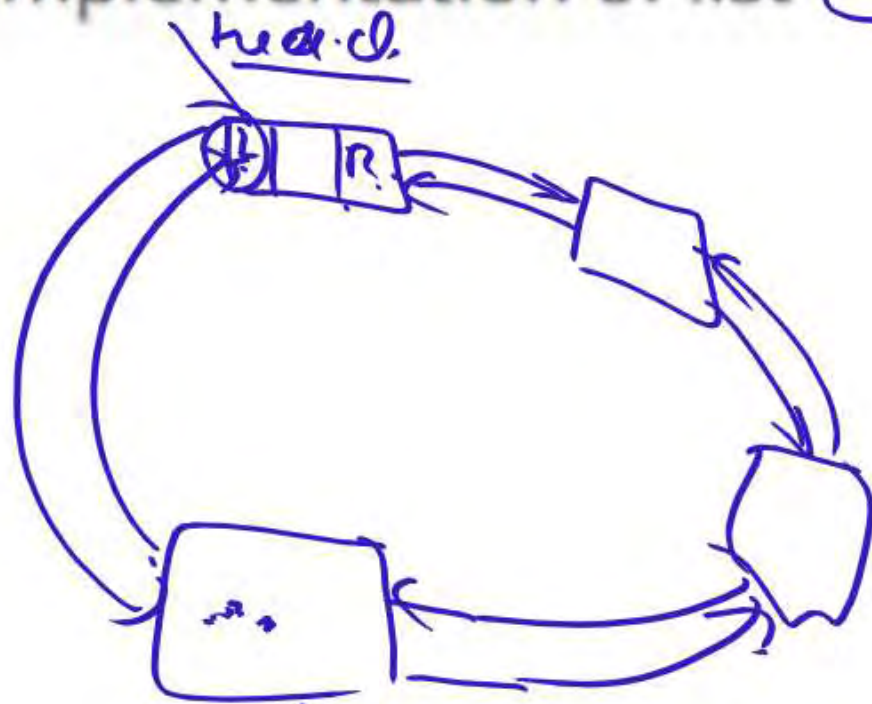
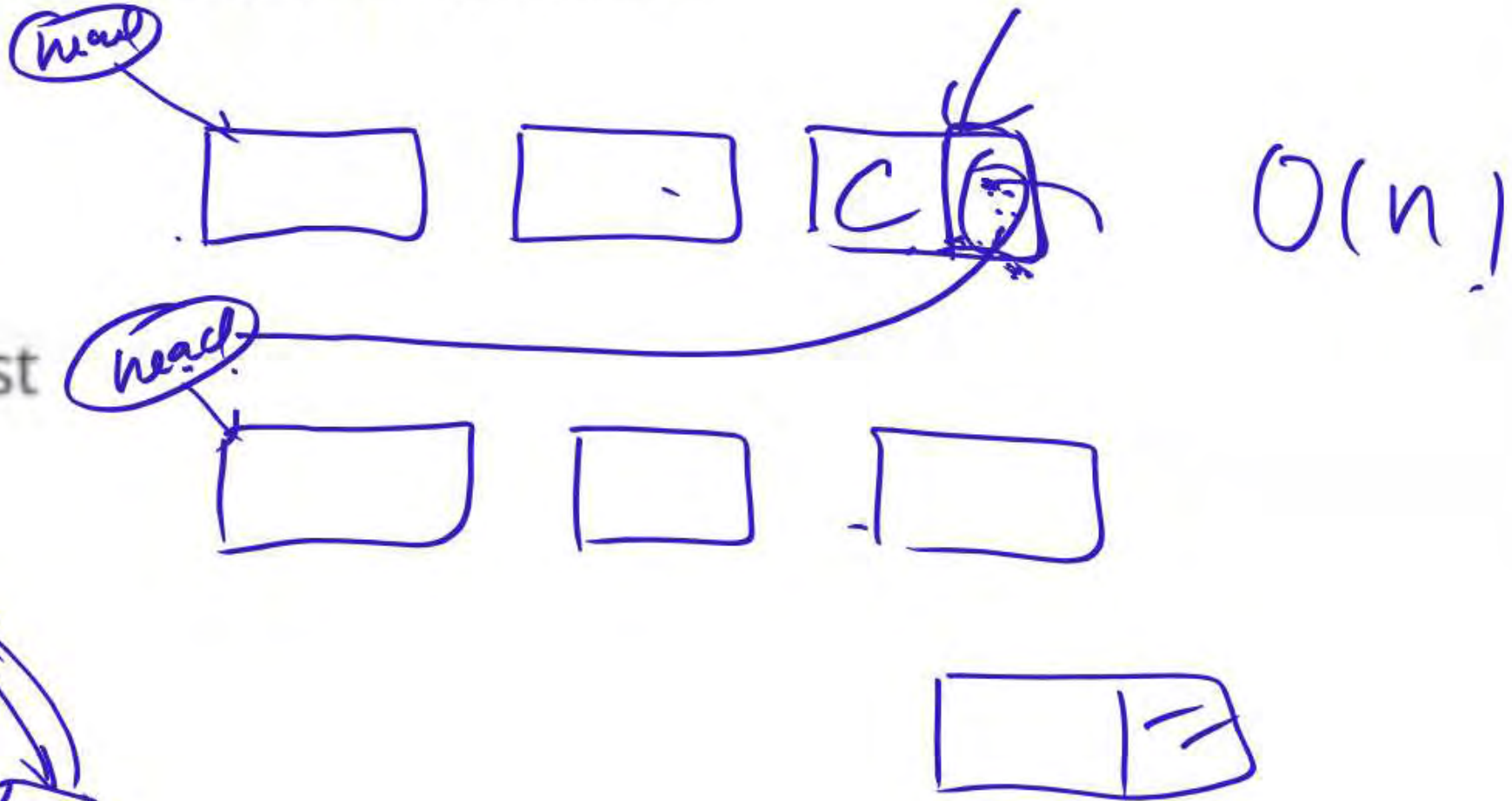


7. What would be the asymptotic time complexity to insert an element at the second position in the linked list?

- a) $O(1)$
- b) $O(n)$
- c) $O(n^2)$
- d) $O(n^3)$

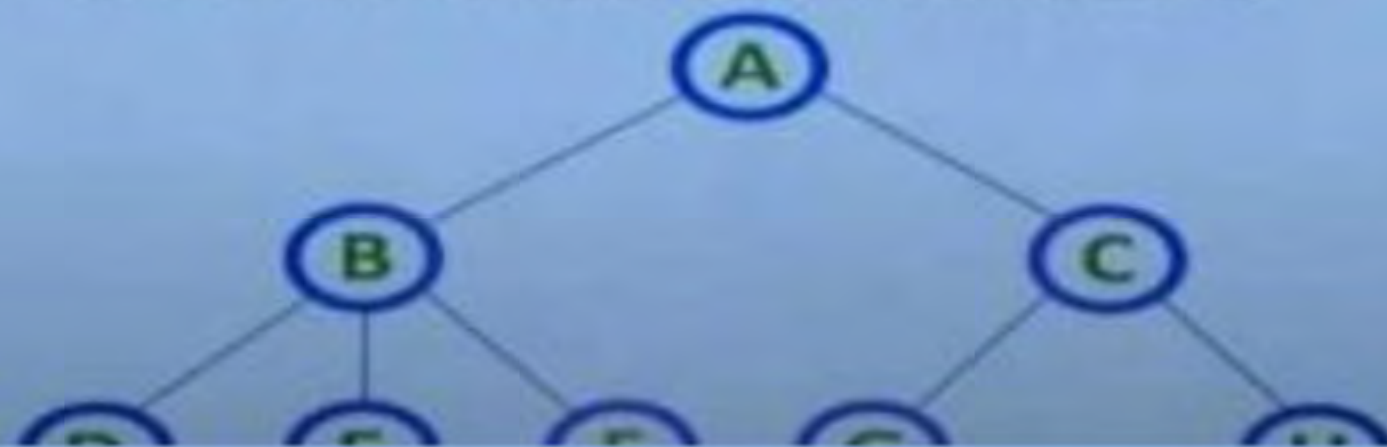
8. The concatenation of two lists can be performed in $O(1)$ time. Which of the following variation of the linked list can be used?

- ☒ a) Singly linked list ☒
- ☒ b) Doubly linked list ☒
- ☒ c) Circular doubly linked list
- ☐ d) Array implementation of list



Data Structure	Access	Insert	Delete	Use Case
Array	$O(1)$	$O(n)$	$O(n)$	Random access, fixed size
Linked List	$O(n)$	$O(1)/O(n)$	$O(1)/O(n)$	Dynamic size, insertion-heavy
Stack	$O(1)$	$O(1)$	$O(1)$	Recursion, parsing
Queue	$O(1)$	$O(1)$	$O(1)$	Task scheduling
Hash Table	$O(1)^*$	$O(1)^*$	$O(1)^*$	Fast lookup by key
Tree (BST)	$O(\log n)$	$O(\log n)$	$O(\log n)$	Sorted data, hierarchy
Heap	$O(n)$	$O(\log n)$	$O(\log n)$	Priority handling
Graph	Varies	Varies	Varies	Network modeling
Trie	$O(L)$	$O(L)$	$O(L)$	Prefix search

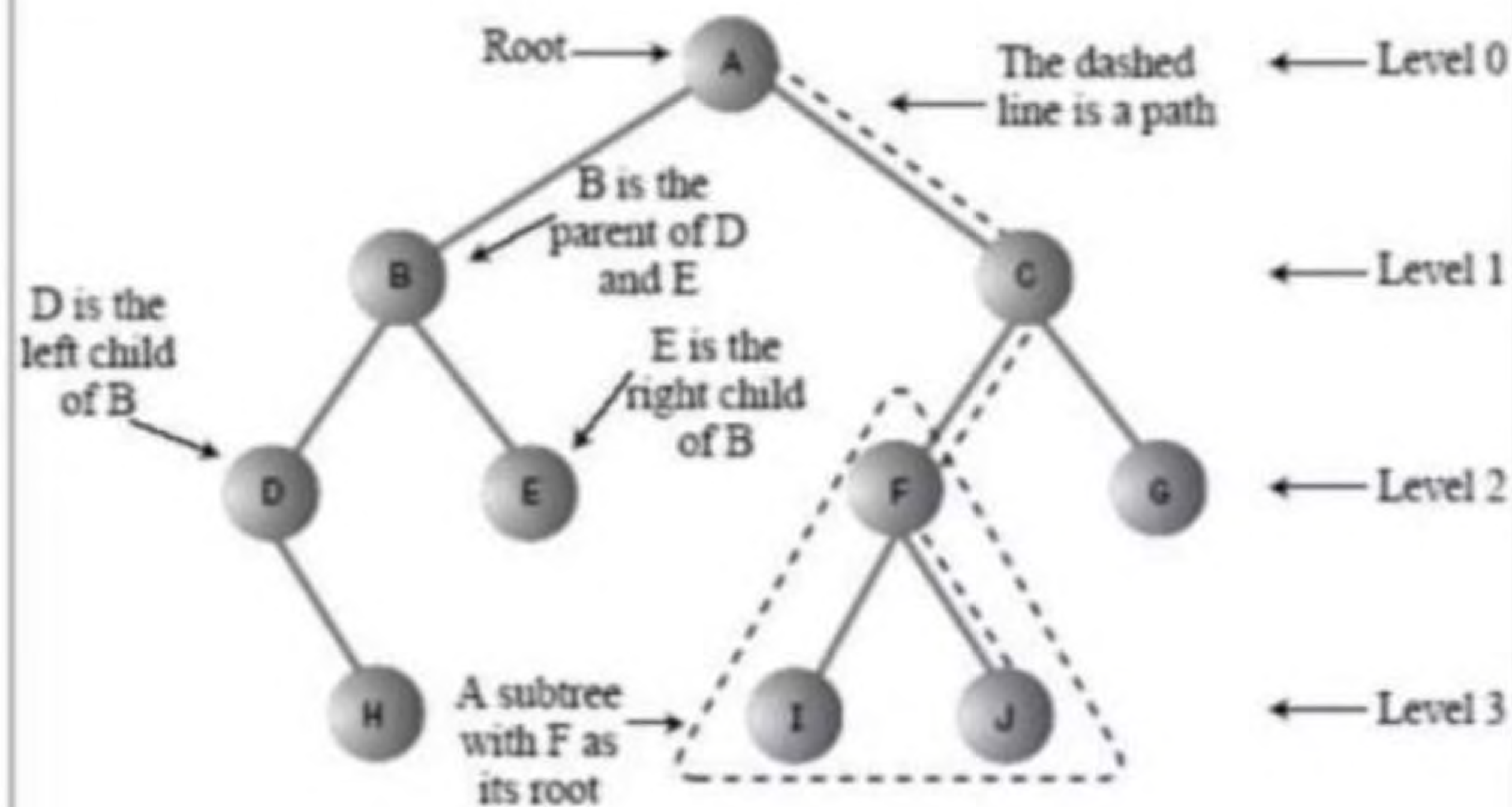
- **Root Node:** The top-most node, serving as the origin of the tree.
- **Edge:** The connection between two nodes. A tree with 'N' nodes has exactly 'N-1' edges.
- **Parent Node:** A node that has one or more children.
- **Child Node:** A node that descends from a parent node.
- **Leaf (External) Node:** A node with no children.
- **Internal Node:** A node with at least one child.
- **Degree of Node:** The number of children a node has.
- **Level/Depth:** Indicates the step count from the root (Level 0) to a particular node.
- **Path:** A sequence of nodes connected by edges.
- **Subtree:** A tree formed from a child node and its descendants.

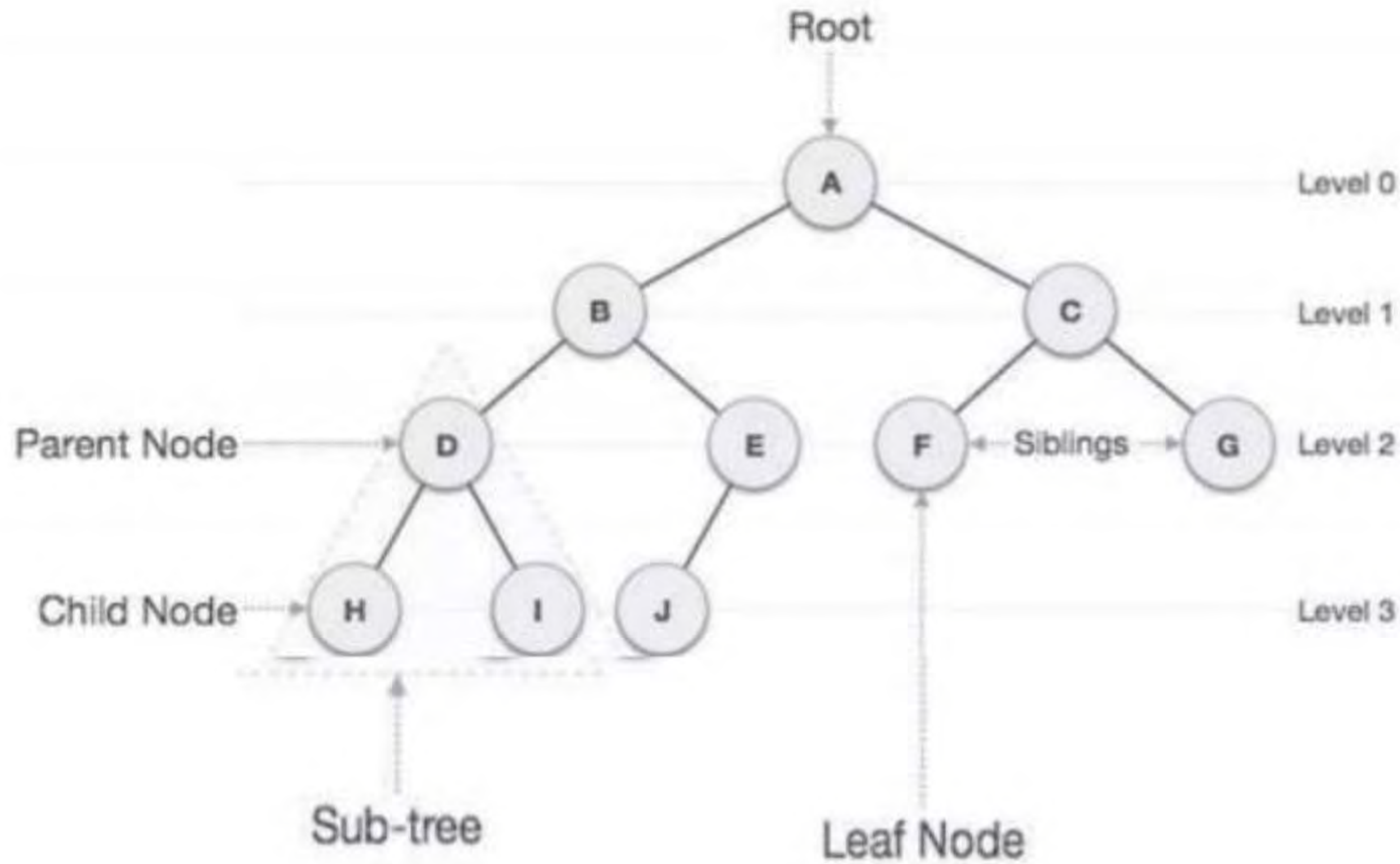


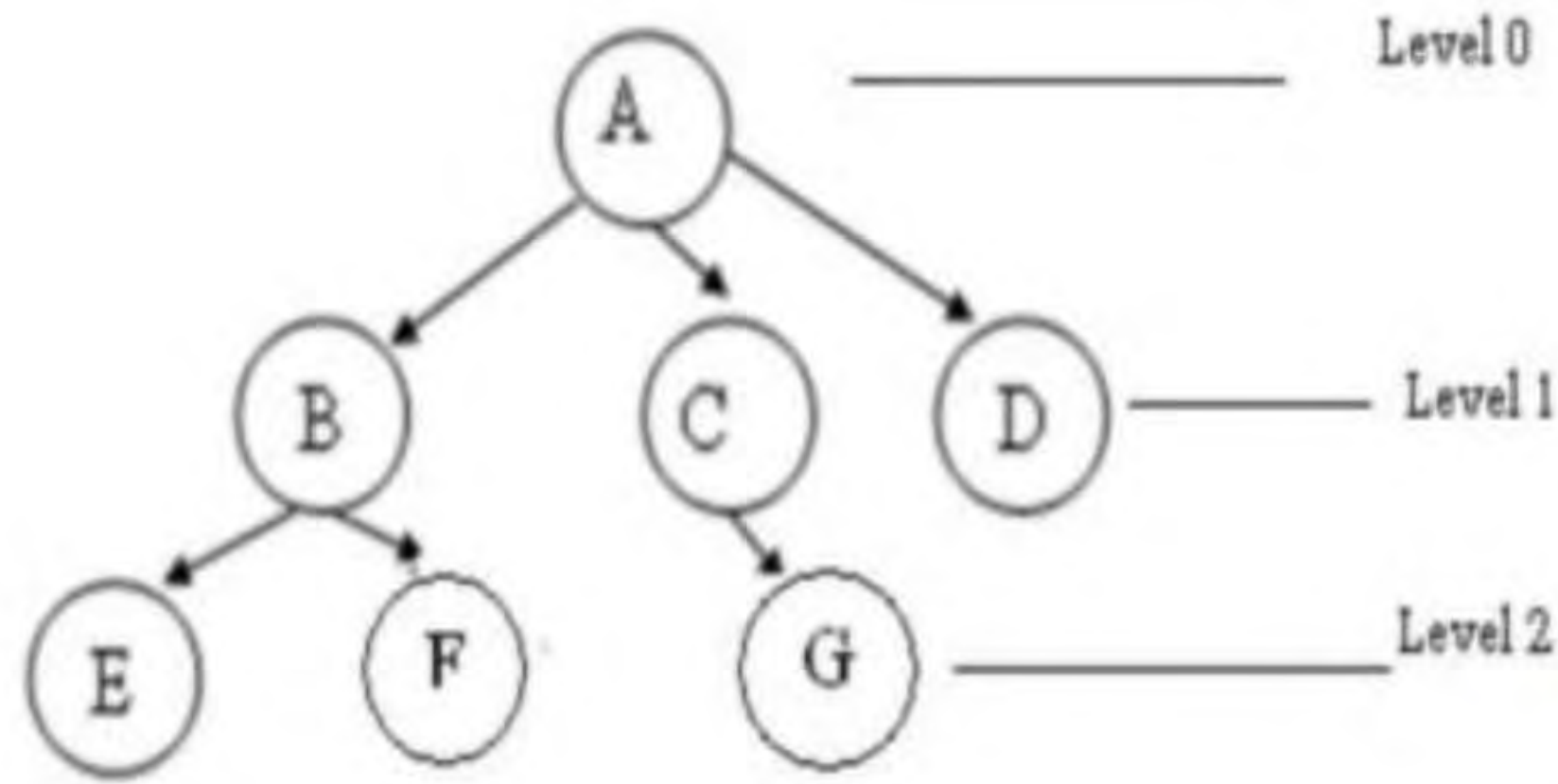
Binary tree

- A **binary tree** is a type of tree data structure where each node can have at most **two children**, commonly referred to as **left** and **right** child nodes. It is structured as a set of nodes with the following properties:
- **Empty Tree:** A binary tree can be empty (no nodes).
- **Root Node:** If not empty, the tree has a unique root node (R).
- **Subtrees:** The remaining nodes are partitioned into two disjoint subtrees, **T1** (left) and **T2** (right).
- Representation of tree in memory
 - Sequential representation – using an array info and left child and right child
 - Linked Representation – using self-referential structure node

```
struct node {  
    int data;  
    struct node* left;  
    struct node* right;  
}
```







- ✓ A is the root node
- ✓ B is the parent of E and F
- ✓ D is the sibling of B and C
- ✓ E and F are children of B
- ✓ E, F, G, D are external nodes or leaves
- ✓ A, B, C are internal nodes
- ✓ Depth of F is 2
- ✓ the height of tree is 2
- ✓ the degree of node A is 3
- ✓ The degree of tree is 3

Characteristics of trees

- Non-linear data structure
- Combines advantages of an ordered array
- Searching as fast as in ordered array
- Insertion and deletion as fast as in linked list
- Simple and fast

Application

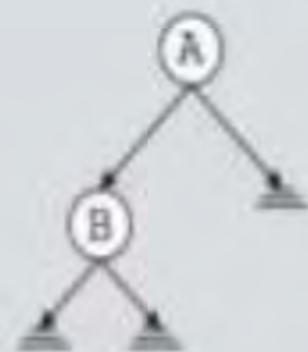
- Directory structure of a file store
- Structure of an arithmetic expressions
- Used in almost every 3D video game to determine what objects need to be rendered.
- Used in almost every high-bandwidth router for storing router-tables.
- used in compression algorithms, such as those used by the .jpeg and .mp3 file-formats.

Binary Trees

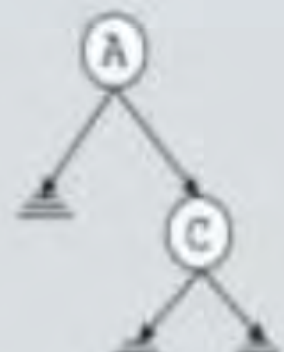
- A **binary tree**, T , is either empty or such that
 - T has a special node called the **root** node
 - T has two sets of nodes L_T and R_T , called the **left subtree** and **right subtree** of T , respectively.
 - L_T and R_T are binary trees.



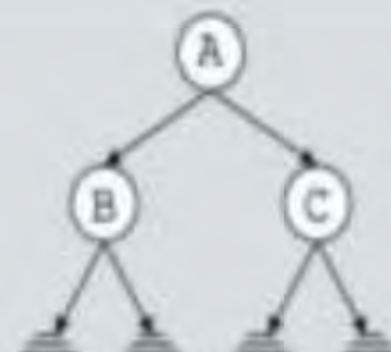
(a) Binary tree
with one node



(b) Binary tree
with two nodes



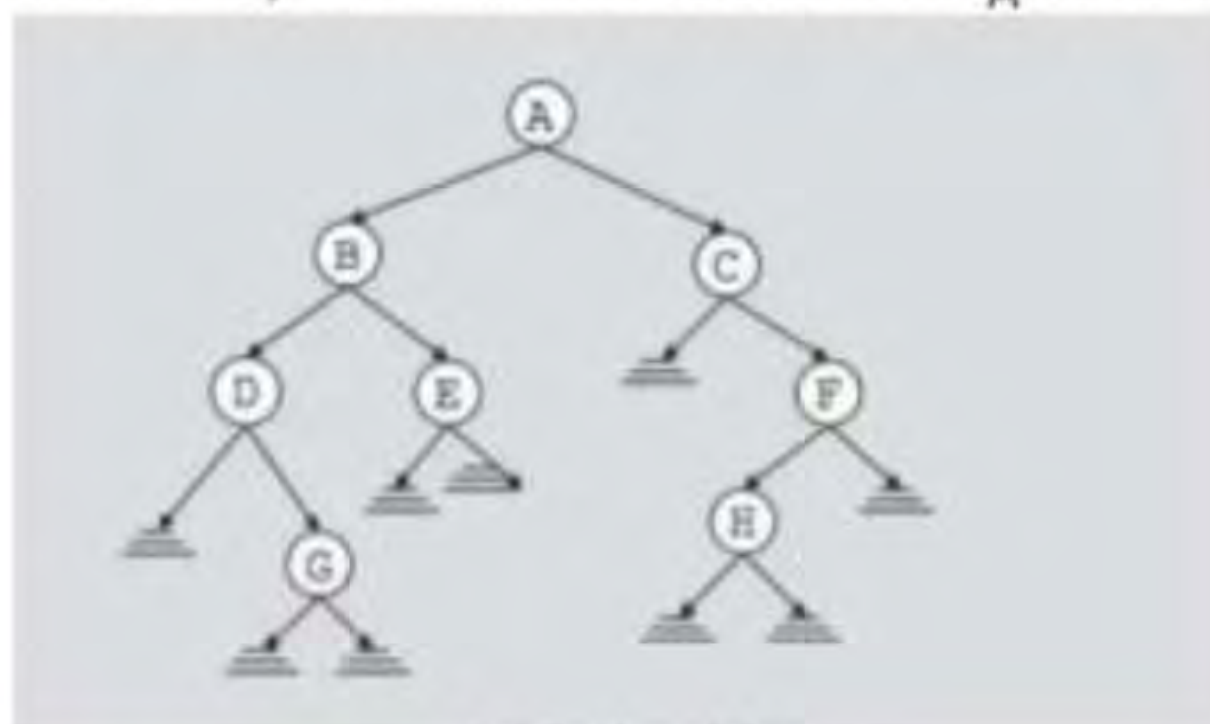
(c) Binary tree
with two nodes



(d) Binary tree
with three nodes

Binary Tree

- The root node of this binary tree is A.
- The left sub tree of the root node, which we denoted by L_A , is the set $L_A = \{B, D, E, G\}$ and the right sub tree of the root node, R_A is the set $R_A = \{C, F, H\}$
- The root node of L_A is node B, the root node of R_A is C and so on



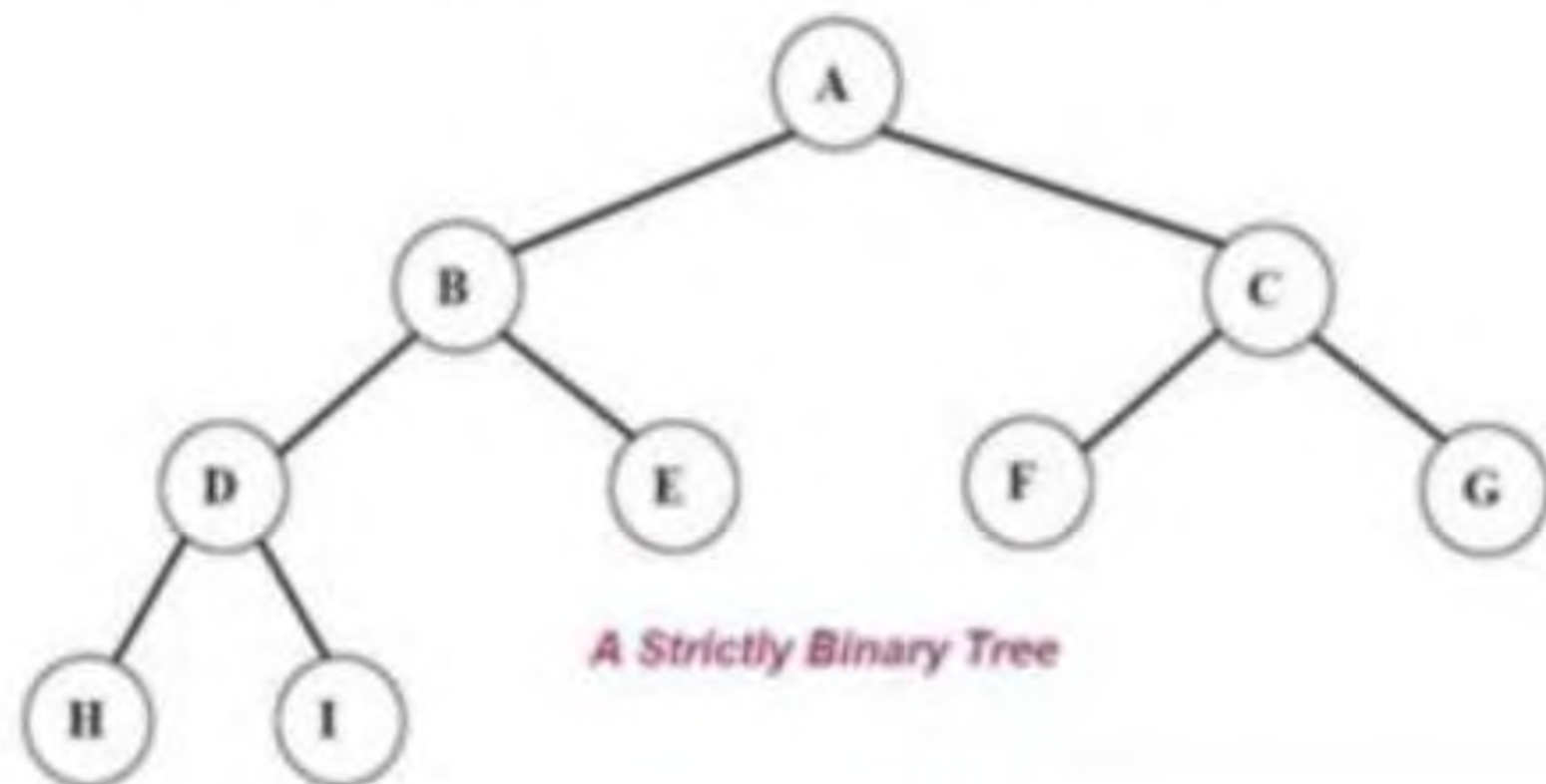
Binary Tree Properties

- If a binary tree contains m nodes at level L , it contains at most $2m$ nodes at level $L+1$
- Since a binary tree can contain at most 1 node at level 0 (the root), it contains at most 2^L nodes at level L .

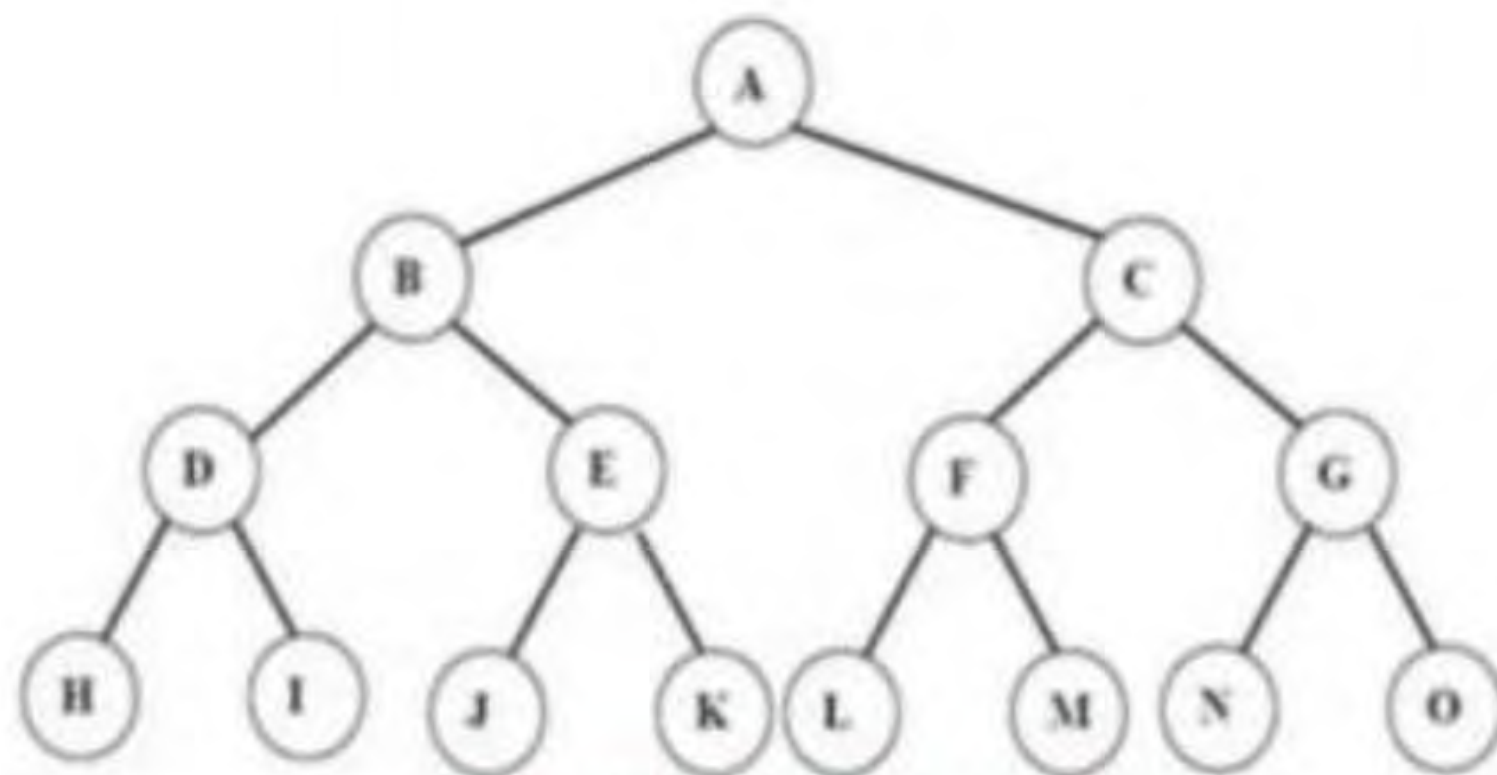
Types of Binary Tree

- Complete binary tree
- Strictly binary tree
- Almost complete binary tree

- Or, to put it another way, all of the nodes in a strictly binary tree are of degree zero or two, never degree one.
- A strictly binary tree with N leaves always contains $2N - 1$ nodes.



- A *complete binary tree* of depth d is called strictly binary tree if all of whose leaves are at level d .
- A complete binary tree has 2^d nodes at every depth d and $2^d - 1$ non leaf nodes



A complete Binary Tree of depth 3

- An almost complete binary tree is a tree where for a right child, there is always a left child, but for a left child there may not be a right child.

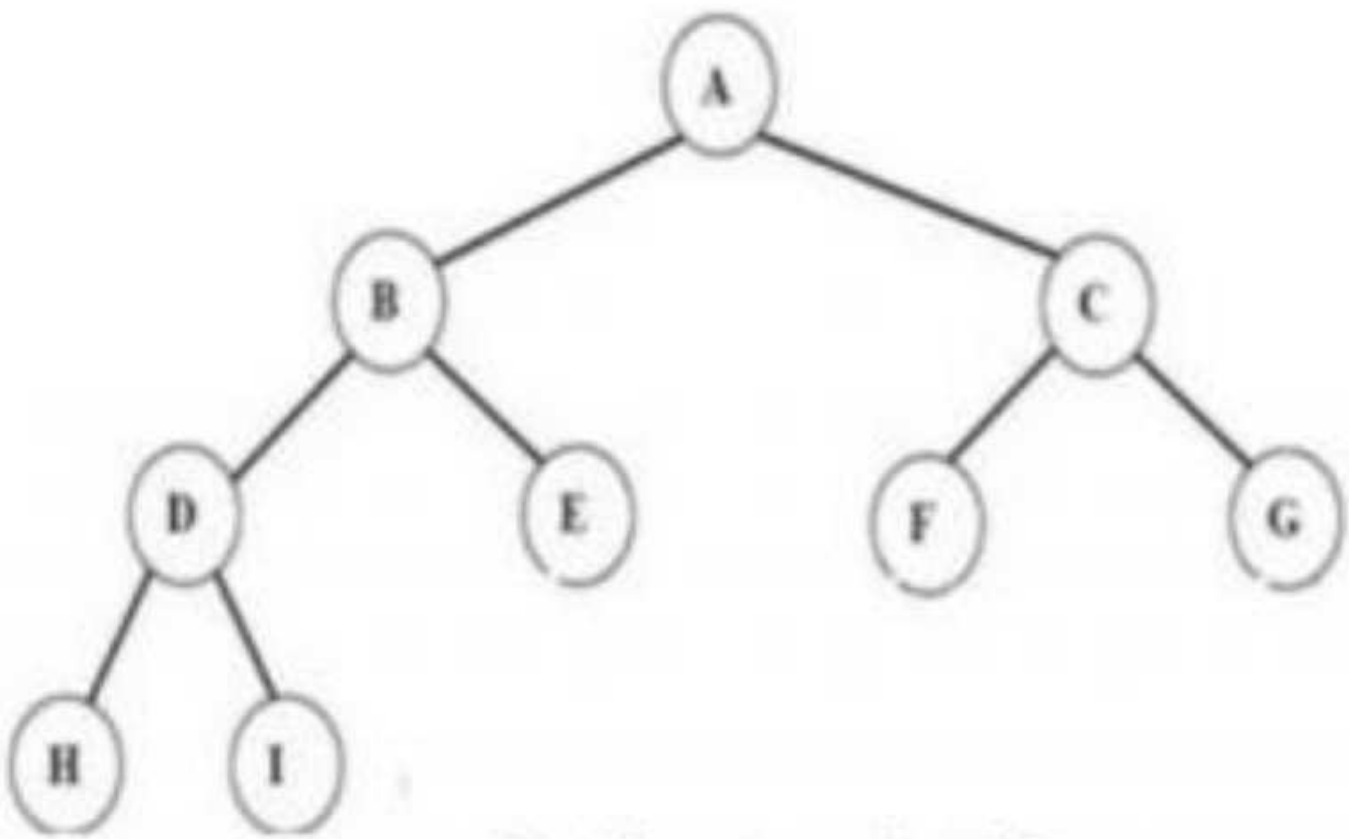


Fig Almost complete binary tree.

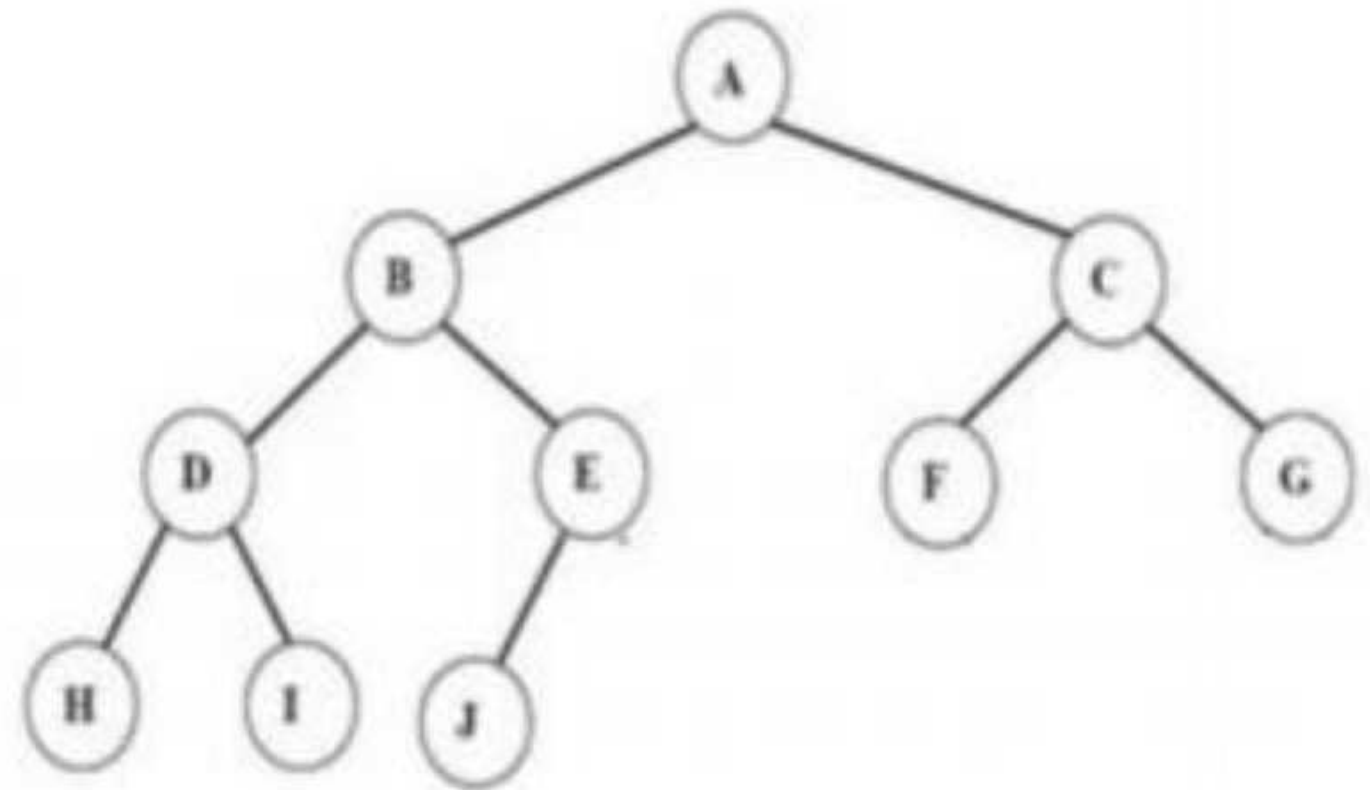


Fig Almost complete binary tree but not strictly !
Since node E has a left son but not a right son.

Operations on Binary tree:

- ✓ **father(n,T):** Return the parent node of the node n in tree T . If n is the root, NULL is returned.
- ✓ **LeftChild(n,T):** Return the left child of node n in tree T . Return NULL if n does not have a left child.
- ✓ **RightChild(n,T):** Return the right child of node n in tree T . Return NULL if n does not have a right child.
- ✓ **Info(n,T):** Return information stored in node n of tree T (ie. Content of a node).
- ✓ **Sibling(n,T):** return the sibling node of node n in tree T . Return NULL if n has no sibling.

- ✓ **Root(T):** Return root node of a tree if and only if the tree is nonempty.
- ✓ **Size(T):** Return the number of nodes in tree T
- ✓ **MakeEmpty(T):** Create an empty tree T
- ✓ **SetLeft(S,T):** Attach the tree S as the left sub-tree of tree T
- ✓ **SetRight(S,T):** Attach the tree S as the right sub-tree of tree T.
- ✓ **Preorder(T):** Traverses all the nodes of tree T in preorder.
- ✓ **postorder(T):** Traverses all the nodes of tree T in postorder
- ✓ **Inorder(T):** Traverses all the nodes of tree T in inorder.

C representation for Binary tree:

```
struct bnode
```

```
{
```

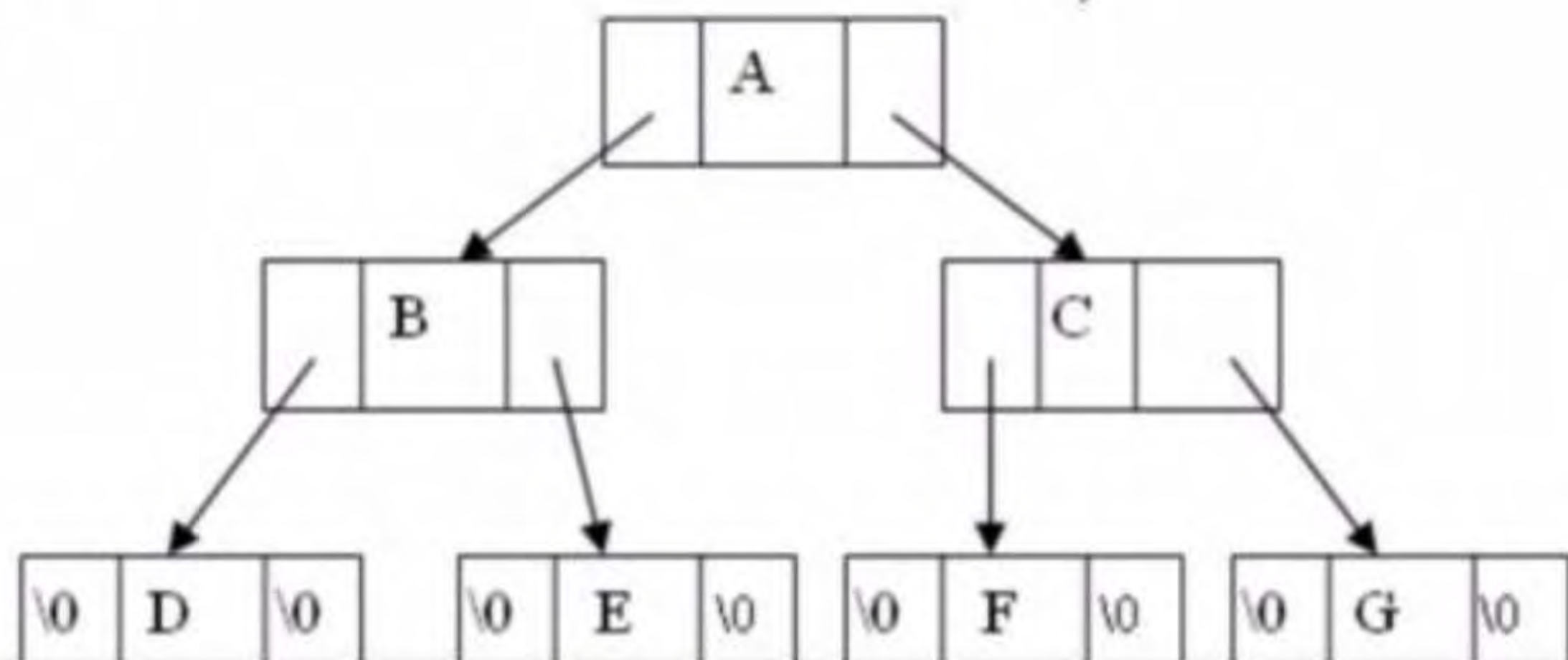
```
    int info;
```

```
    struct bnode *left;
```

```
    struct bnode *right;
```

```
};
```

```
struct bnode *root=NULL
```



Tree traversal

- Traversal is a process to visit all the nodes of a tree and may print their values too.
- All nodes are connected via edges (links) we always start from the root (head) node.
- There are three ways which we use to traverse a tree
 - In-order Traversal
 - Pre-order Traversal
 - Post-order Traversal

- Pre-order

<root><left><right>

- In-order

<left><root><right>

- Post-order

<left><right><root>

- The preorder traversal of a nonempty binary tree is defined as follows:
 - Visit the root node
 - Traverse the left sub-tree in preorder
 - Traverse the right sub-tree in preorder

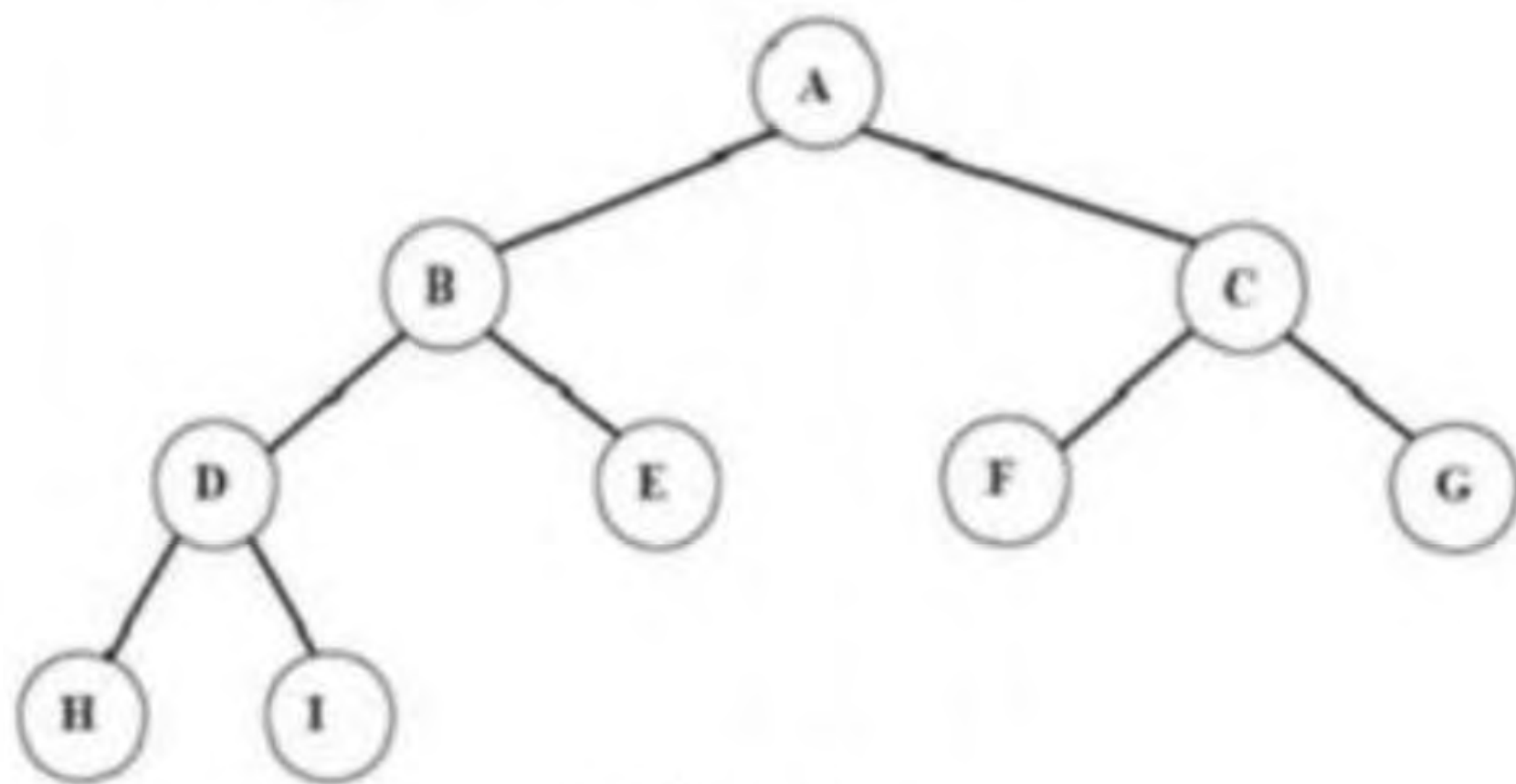


fig Binary tree

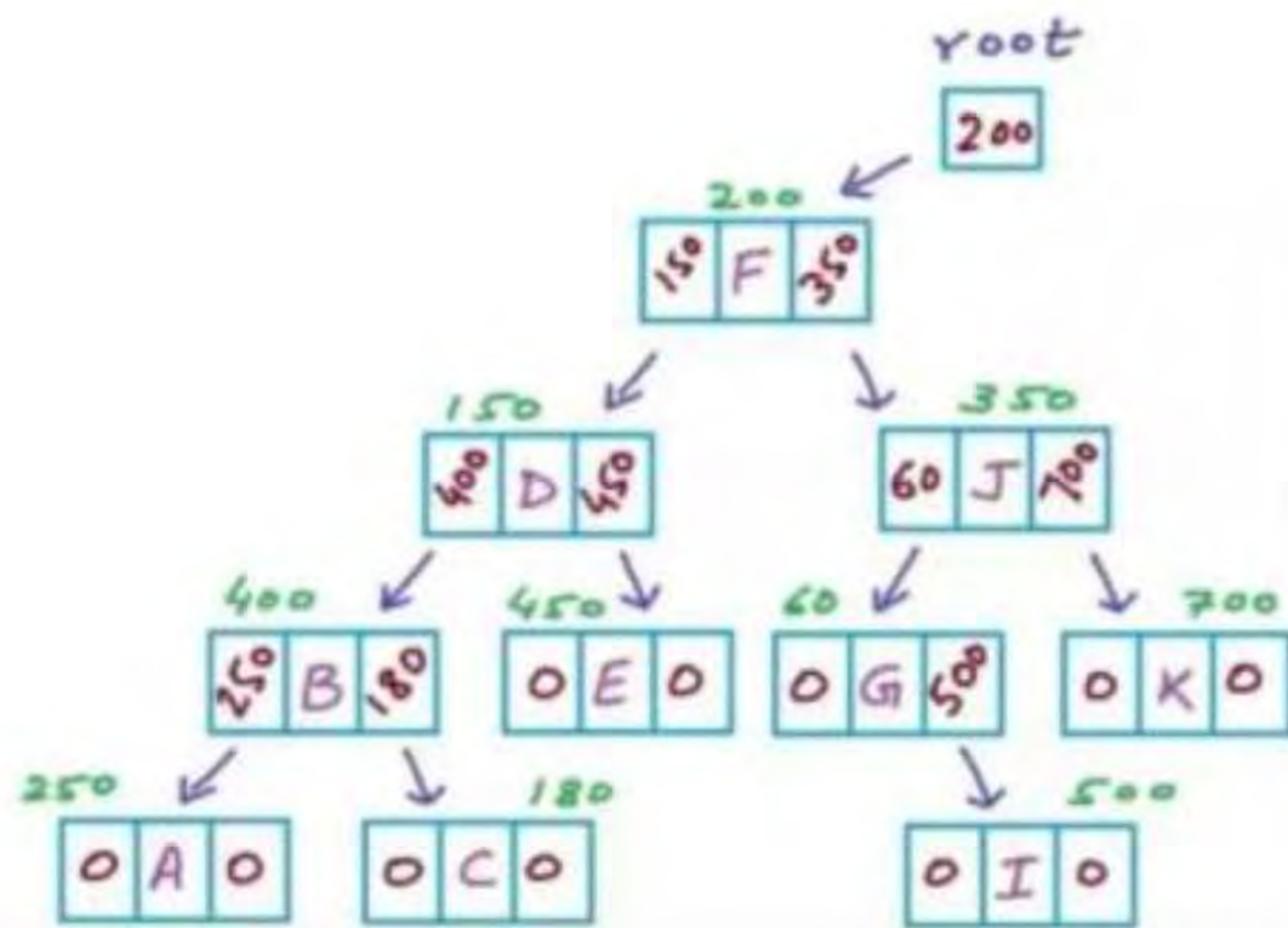
The preorder traversal output of the given tree is: A B D H I E C F G

Pre-order Pseudocode

```

struct Node{
    char data;
    Node *left;
    Node *right;
}

void Preorder(Node *root)
{
    if (root==NULL) return;
    printf ("%c", root->data);
    Preorder(root->left);
    Preorder(root->right);
}
    
```



- The in-order traversal of a nonempty binary tree is defined as follows:
 - Traverse the left sub-tree in in-order
 - Visit the root node
 - Traverse the right sub-tree in inorder

- The in-order traversal output of the given tree is
H D I B E A F C G

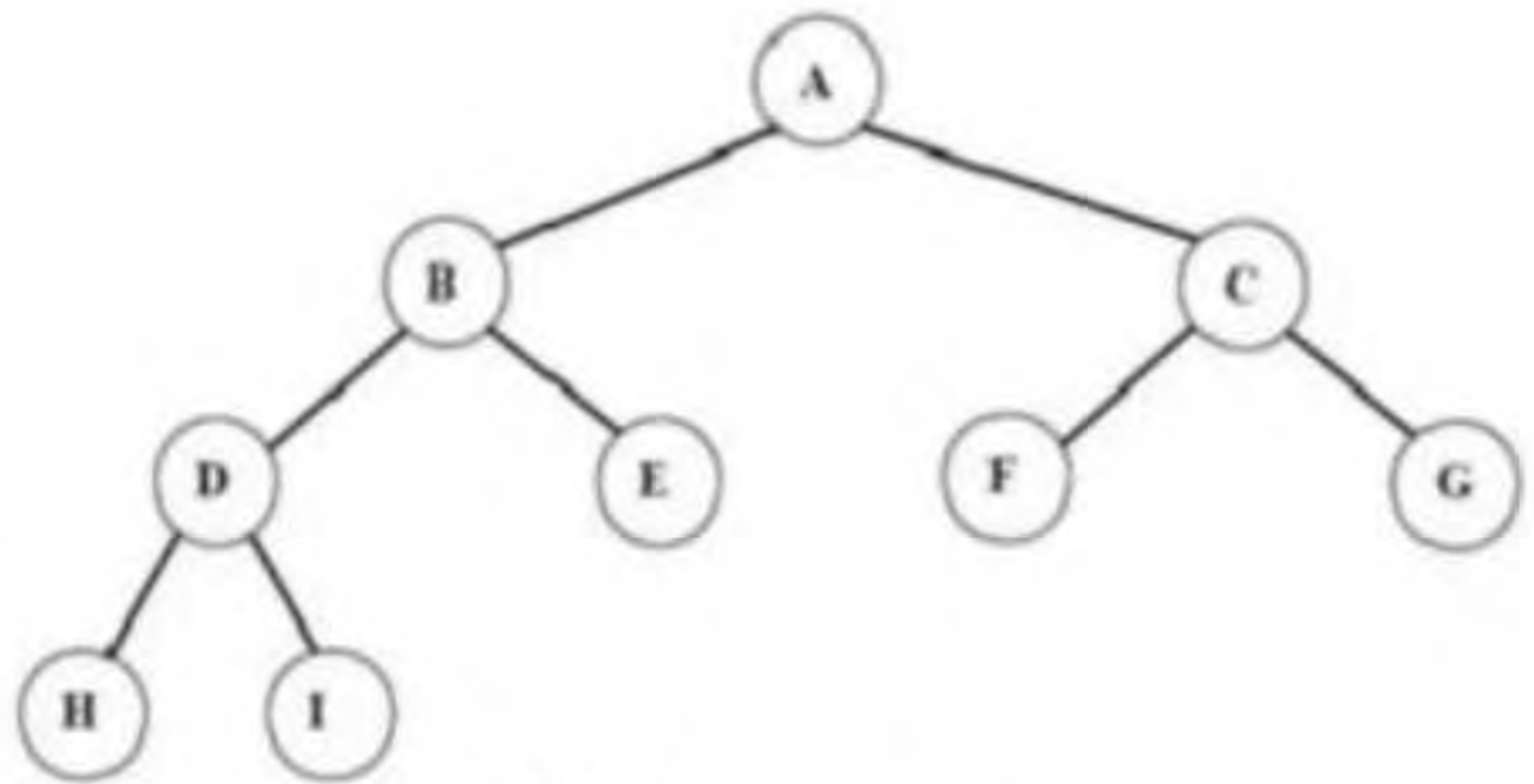


fig Binary tree



2 mins Summary



TEACHING
WALLAH

Topic

linked List

THANK YOU